X-MHRDS Project Playbook

Master Technical Playbook & Professor Presentation Guide

X-MHRDS — Explainable Mental Health Risk Detection System

Table of Contents

(Auto-generated field — in Word, right-click it and choose "Update Field" → "Update entire table" if it shows as empty when you first open this file.)

X-MHRDS — Explainable Mental Health Risk Detection System

Master Technical Playbook & Professor Presentation Guide

How this document was produced: every claim below was verified against the actual source files in this repository (not inferred from naming conventions or aspiration), and the headline numbers (model metrics, construct-validity figures) were re-confirmed by calling the live backend during this session. Where the implementation is a deliberate simplification or a known limitation, that is stated explicitly rather than glossed over — a professor who opens the code should find it matches this document exactly.

Table of Contents

---

1. Executive Summary

X-MHRDS is a research prototype — explicitly *not* a clinical instrument — that classifies short social-media-style posts for suicide-risk language, and wraps that classification in a stack of explainability, fairness, robustness, calibration, and clinical-decision-support tooling. It is built around one central idea: **a single accuracy number is not enough for a system that touches mental health**, so almost every service in this codebase exists to answer a follow-up question about the classifier rather than to improve its raw accuracy.

The system has two halves:

- **`ai_model/` + `backend/src/services/`** — a Python/FastAPI backend: four trained classifiers, four explainability methods, a fairness auditor, a construct-validity auditor, a robustness suite, drift detection, calibration, out-of-distribution detection, and a set of "clinical copilot" services (anonymization, emotion/cognitive-distortion tagging, DSM-5/C-SSRS-grounded response drafting, semantic case search, multilingual analysis, live monitoring).
- **`frontend/`** — a React 19 + Vite single-page app with nine feature tabs, a landing page, a command palette, and a light/dark theme system, talking to the backend over a REST API and one WebSocket.

Four classifiers are trained side by side, deliberately spanning the interpretability/accuracy trade-off:

Model	Representation	Test Accuracy	Test F1
Logistic Regression	TF-IDF (uni+bigram, 5,000 features)	91.38%	91.20%
SVM (Calibrated LinearSVC)	TF-IDF (uni+bigram, 5,000 features)	91.91%	91.84%
BERT (`bert-base-uncased`, fine-tuned)	WordPiece, 128-token context	97.69%	97.68%
RoBERTa (`roberta-base`, fine-tuned)	Byte-pair encoding, 128-token context	97.64%	97.65%

(Figures above were pulled live from `GET /api/metrics` during this session — see [Section 9](#9-verified-evaluation-results).)

---

2. Problem Statement & Motivation

Suicide-risk text classification is a well-studied NLP problem, but most published systems stop at "here is a probability." That is a poor fit for a clinical-adjacent use case for three reasons this project explicitly designs around:

29. **A wrong prediction is not equally costly in both directions.** A false negative on active suicidal ideation is far worse than a false positive. The system therefore never collapses to a single binary output — every prediction is escalated through a 4-tier severity model, and every transformer prediction carries a calibrated confidence (what/why/how: 26.8) plus an out-of-distribution flag (what/why/how: 26.12), so a human reviewer knows *how much* to trust it, not just *what* it says.
30. **"The model is 97% accurate" does not tell a clinician* why *it flagged a specific post.** Hence the four-explainer XAI suite (SHAP, Integrated Gradients, LIME, Leave-One-Out — what/why/how for each: Section 26.7) with a convergence-correlation matrix, so a reviewer can see the actual trigger words and check whether independent explanation methods agree.
31. **A model can be accurate for the wrong reasons** — e.g. by detecting generic sadness rather than suicide-specific risk language, or by working worse for some ways of phrasing distress than others. Hence the construct-validity auditor (26.11) and the fairness auditor (26.10) described in Section 13.

The project's own framing (see the in-app landing page, `frontend/src/components/landing/LandingPage.jsx`) is explicit that the delivered system exceeds its original proposal scope in exactly these trust/governance areas — cognitive-distortion tagging, energy-based OOD detection, MC-Dropout uncertainty, live per-user escalation tracking, and the clinical safety copilot were all built beyond the initial plan.

---

3. Literature Review & Theoretical Grounding

A structured literature review of 16 papers (all 2026 arXiv preprints on explainable/robust/fair mental-health NLP) lives at `literature_review/literature_review.md` (also exported as `literature_review.docx`, with 14 of the 16 source PDFs archived locally under `literature_review/papers/` — 2 are paywalled and represented as placeholders). This review is the direct theoretical source for several concrete design decisions in the code, not just background reading:

- *Dehghan & Ashrafi (2026),* Auditing Construct Overlap in Explainable Machine Learning* **(arXiv:2607.10633) is cited** by name in a code comment** in `backend/src/services/construct_validity_auditor.py`. Their residualization protocol (regress the model's prediction on a confound, check whether the residual still correlates with the true label) is implemented directly — see Section 13.2.
- **Liu et al. (2020)**, the energy-based OOD detection paper, is the basis for `TransformerClassifier.compute_energy()` in `ai_model/src/transformer_models.py` — see Section 11.

- **Anikejeva & Sirts (2026),* Exploring Profiles of Cognitive Distortions*** (arXiv:2605.24996) and the broader CBT literature motivate `cognitive_distortion_analyzer.py`'s Beck/Burns taxonomy implementation.
- **Belcastro et al. (2026),* Explainable Detection of Depression Status Shifts*** (arXiv:2605.14995) — temporal trajectory profiling with change-point detection — is the closest published analogue to this project's Live Monitor escalation-trend engine (Section 17), though this project's change-point method (`ruptures` Binary Segmentation) differs from their offline LLM-summarization approach.
- Several reviewed papers (Loweimi et al. 2026 on LLM screening reliability; Hussain et al. 2026 on hallucination detection in mental-health chatbots) directly informed the **safety-first design of the Gemini integration** (Section 15): the reviewed literature consistently finds that LLM-as-judge/LLM-as-clinician approaches are unreliable when given free rein, which is why this system never lets an LLM decide a risk tier or protocol — it only narrates a decision that deterministic code already made.

The other 12 papers (federated multimodal depression detection, RL-aligned LLM reasoning, on-device zero-egress psychiatric AI, speech-based fairness audits, etc.) establish the broader state of the field but do not map to a specific line of code — cite them as evidence you surveyed the space, not as implementation sources.

---

4. Dataset

Source: the Suicide Watch dataset on Kaggle — Reddit posts from `r/SuicideWatch` (label `suicide`) paired with posts from non-crisis subreddits (label `non-suicide`).

- **Raw size:** 232,074 posts, exactly class-balanced (116,037 / 116,037).
- **Subsampling:** `config.SAMPLE_SIZE = 15000` (`backend/src/config/settings.py`). `preprocessing.py` draws up to 7,500 posts per class (`sample_size // 2`), preserving balance.
- **Split:** stratified, `TEST_SPLIT = 0.15`, `VAL_SPLIT = 0.15` (of the *total*, so the val split is computed as `0.15 / (1 - 0.15)` of the train+val remainder). Net result: **≈10,500 train / ≈2,250 val / ≈2,250 test**, all class-balanced. Random state is fixed at `RANDOM_STATE = 42` throughout for reproducibility.
- **Average length:** ≈131 words per cleaned post (reported on the in-app landing page).
- **No synthetic fallback:** `preprocess_dataset()` raises `FileNotFoundError` if the raw CSV isn't present at `data/Suicide_Detection.csv` — the pipeline will not silently fabricate data.
- **EDA artifacts:** `data/processed/label_distribution.png` (class countplot) and `data/processed/word_count_distribution.png` (word-count histogram, clipped to the 95th percentile) are generated by `generate_eda_plots()` in `preprocessing.py`.

Important scale correction: the dataset card sometimes gets described (including in an earlier draft of this very document) as if the *full* 232k-post corpus is what gets split 80/10/10 and trained on. That is not what the code does. Only the 15,000-post subsample is ever used for training/evaluation — say so if asked, since a professor may compute "185,659 training rows" and find it doesn't match anything in `data/processed/train.csv`.

5. Preprocessing Pipeline

`backend/src/utils/preprocessing.py`, `clean_text()`:

32. **PII masking first, before lowercasing** — calls `services/anonymizer.py`'s `mask_pii()` on the *original-case* text, because the name-detection heuristic (`"I am John"` → `"I am [NAME]"`) depends on capitalization to distinguish proper nouns from ordinary lowercase words.
33. Lowercase.
34. Strip URLs (`https?://\S+|www\.\S+`).
35. Strip HTML tags.
36. Strip Reddit `u/` and `r/` mentions not already caught by the anonymizer.
37. Strip everything that isn't a-z, whitespace, or `[/]` (the PII placeholder brackets are deliberately preserved).
38. Collapse repeated whitespace.

PII anonymizer (`services/anonymizer.py`, `mask_pii()`) — pure regex, five categories: `r/subreddit` → `[SUBREDDIT]`, `u/username` → `[USER]`, emails → `[EMAIL]`, phone numbers (several formats) → `[PHONE]`, and a narrow name-detection heuristic (`"my name is X"` / `"I am X"` / `"I'm X"` where X is capitalized) → `[NAME]`. This is intentionally simple and rule-based — it is **not** a named-entity-recognition model, so it will miss names that don't follow one of those two sentence patterns. This is the same function used both in the offline preprocessing pipeline and live at inference time (`sandbox.py`'s `/analyze` route, gated by the `anonymize_active` flag).

Label mapping: `suicide → 1`, `non-suicide → 0` throughout the codebase — label `1` always means "risk" / the positive class.

6. Modeling: The Four Classifiers

6.1 Why four models, and why these four

The explicit design rationale (also stated on the landing page) is to benchmark the **interpretability–accuracy trade-off**, not to chase a single best leaderboard number:

- **Logistic Regression** and **SVM** over TF-IDF give a *linear*, coefficient-level explanation for free (`explain_baseline()` in `explainability.py` literally multiplies the TF-IDF value by the model's learned coefficient per token) — maximally transparent, cheap to run on CPU, easy to audit.
- **BERT** and **RoBERTa** (both fine-tuned, not zero-shot/prompted) give much higher raw accuracy by capturing context the bag-of-words models structurally cannot (negation, sarcasm, multi-word idioms), at the cost of needing post-hoc explanation methods (SHAP, Integrated Gradients, LOO) rather than an exact closed-form one.

Running all four side by side, on the same data splits, lets the system's own Multi-XAI Studio and Analytics tab make an evidence-based case for *how much* accuracy the transformers buy versus how much interpretability the linear models keep — rather than asserting it.

For a plain-language what-is-it/why/how of TF-IDF, Logistic Regression, Linear SVM, and Transformers/BERT/RoBERTa themselves (not just how this project configured them), see [Section 26.1–26.3](#261-tf-idf-term-frequencyinverse-document-frequency).

6.2 Baseline models — `ai_model/src/baseline_models.py`

- **Vectorizer:** `TfidfVectorizer(max_features=5000, ngram_range=(1, 2))` — unigrams and bigrams, top 5,000 by TF-IDF weight, fit once on the training split and reused for val/test/inference.
- **Logistic Regression:** `sklearn.linear_model.LogisticRegression(max_iter=1000, random_state=42)`.
- **SVM:** `sklearn.svm.LinearSVC(C=1.0, random_state=42)` wrapped in `CalibratedClassifierCV(base_svm, cv=3)` — LinearSVC has no native `predict_proba`, so 3-fold cross-validated Platt/sigmoid calibration is used specifically to *get* probabilities out of it (this is a mechanical necessity, distinct from the temperature-scaling calibration layer described in Section 8, which is applied afterwards on top of both baselines *and* the transformers).
- **Inference wrapper:** `BaselineClassifier` loads the pickled vectorizer + model and exposes `.predict_proba(text) -> {"prediction": int, "probabilities": [p0, p1]}`, the same interface every other classifier in the codebase implements, which is what lets one `CalibratedClassifier` proxy and one set of explainability functions work across all four models.

6.3 Transformer models — `ai_model/src/transformer_models.py`

- **Models:** `bert-base-uncased` and `roberta-base` (Hugging Face `transformers`, `AutoModelForSequenceClassification`, `num_labels=2`), loaded via `AutoTokenizer`/`AutoModelForSequenceClassification.from_pretrained`.
- **Tokenization:** `max_length=128`, truncation + padding to max length.
- **Optimizer:** `AdamW`, `lr=2e-5`.
- **Schedule:** cosine schedule with warmup (`get_cosine_schedule_with_warmup`), warmup steps = `WARMUP_RATIO (0.1) × total_steps`.
- **Epochs:** 3 (full run) or 1 (`--quick` smoke-test mode, which also subsamples to 100 train / 30 val / 30 test rows).
- **Batch size:** 16.
- **Mixed precision:** `torch.amp.GradScaler`, enabled automatically when a CUDA device is available (`USE_MIXED_PRECISION = True`); falls back to full precision on CPU.
- **Gradient clipping:** max norm 1.0, applied every step (before the optimizer step, after unscaling in the AMP path).
- **Exponential Moving Average (EMA) of weights**, decay `0.999` — a full custom `EMA` class maintains a shadow copy of every trainable parameter and updates it after every optimizer step. **Both validation-time model selection and the final saved checkpoint use the EMA-averaged weights, not the raw last-step weights** — the code includes stash/restore logic (`evaluate_with_ema`, `save_with_ema_weights`) so evaluating/saving the EMA snapshot never disturbs the live training weights, letting training continue uninterrupted.

- **Checkpoint selection:** best **validation F1** (not accuracy) is saved — a deliberate choice, since accuracy alone can be inflated by a model that leans on the majority class.
- **Device:** CUDA if available, else CPU (`torch.device("cuda" if torch.cuda.is_available() else "cpu")`).

Why the cosine schedule, mixed precision, gradient clipping, and EMA are all needed together — not just what each one is — is explained in [Section 26.5](#265-fine-tuning-stabilizers-cosine-warmup-mixed-precision-gradient-clipping) and [26.6](#266-exponential-moving-average-ema-of-weights).

6.4 Multi-tier classifier — `ai_model/src/multi_tier_classifier.py`

See Section 10 — it is not just a set of thresholds; it has an ML-driven primary path and a threshold-rule fallback path, and the two are frequently conflated, so they get their own section.

---

7. Training Procedure in Detail

Reproducible end-to-end sequence (see Section 22 for exact commands):

39. `preprocessing.py` → cleans + PII-masks + splits the raw CSV, saves `train.csv`/`val.csv`/`test.csv` + two EDA plots.
40. `baseline_models.py` → fits the shared TF-IDF vectorizer, trains LR and calibrated-SVM, evaluates on the **test** split, and separately fits temperature-scaling calibration on the **validation** split (`fit_and_save_calibration`) — test and calibration data are kept disjoint on purpose.
41. `transformer_models.py --model bert` / `--model roberta` → fine-tunes for 3 epochs, evaluates on test, fits temperature scaling on val, and calibrates an energy-based OOD threshold from the val-set energy distribution (95th percentile) — again all using validation data only, never test, to avoid contaminating the numbers being reported as "held-out performance."
42. (Optional, run manually / not part of the four scripts above) `robustness.py`, `construct_validity_auditor.py`, `multi_tier_classifier.py` (to train the ML multi-tier head) each load whichever of the four base models are already trained and produce their own `.pkl` artifact under `models/`.

Every one of these scripts is independently runnable and idempotent — re-running `baseline_models.py` retrains from scratch and overwrites its artifacts; nothing here is incremental/continual training.

---

8. Probability Calibration

A raw classifier's `predict_proba` output is not necessarily a trustworthy probability — a model can be systematically over- or under-confident. `ai_model/src/calibration.py` implements **temperature scaling** (Guo et al.–style, one scalar parameter per model):

- `compute_ece(probs, labels, n_bins=10)` — binned **Expected Calibration Error**: the weighted average gap between predicted confidence and observed accuracy across 10 confidence bins.
- `fit_temperature(probs, labels)` — finds a scalar `T` minimizing negative log-likelihood of `sigmoid(logit(probs) / T)` against true labels, via `scipy.optimize.minimize_scalar` bounded to `[0.05, 10.0]`.
- `apply_temperature(probs, T)` — rescales a probability by dividing its logit by `T` and re-applying the sigmoid.

This is fit **once per model, on the validation split, after training** — never on test data — and the fitted temperature + before/after ECE are saved to `models/{model}_calibration.pkl`. At inference time, `sandbox.py`'s `CalibratedClassifier` is a transparent proxy that wraps whichever base classifier was selected and applies the stored temperature to every `predict_proba` call, forwarding all other attribute access straight through (so explainability/OOD/uncertainty code, which needs `.model`/`.tokenizer`/`.device`, keeps working unmodified).

Correction vs. an earlier draft of this document: this is temperature scaling (one global scalar, fit by NLL minimization on logits), not Platt scaling / isotonic regression. The *only* place classical Platt-style calibration appears in this codebase is `CalibratedClassifierCV`'s internal handling of `LinearSVC`, which exists purely to produce a `predict_proba` method for an SVM that doesn't natively have one — it is not the same calibration layer that all four models share.

Full what-is-calibration/why-it-matters/how-the-math-works explanation: [Section 26.8](#268-temperature-scaling--expected-calibration-error-ece).

---

9. Verified Evaluation Results

These numbers were fetched live from the running backend during this session (`GET /api/metrics`), not read from a stale file:

Model	Accuracy	Precision	Recall	F1
Logistic Regression	91.38%	93.14%	89.33%	91.20%
SVM (Calibrated LinearSVC)	91.91%	92.67%	91.02%	91.84%
BERT (fine-tuned)	97.69%	98.12%	97.24%	97.68%
RoBERTa (fine-tuned)	97.64%	97.60%	97.69%	97.65%

These match the figures presented on the in-app landing page, which is a good consistency check that the shipped model artifacts under `models/` are the ones the landing page's claims describe.

Robustness (from the landing page's reported robustness run, BERT): F1 under typo injection = 95.85% (−1.83 points from clean); F1 with an appended distracting sentence = 97.37% (−0.31 points). See Section 13.3 for methodology.

---

10. Multi-Tier Risk Escalation Engine

Binary suicide/non-suicide is too coarse for triage. `MultiTierClassifier` (`ai_model/src/multi_tier_classifier.py`) maps every prediction to one of four tiers:

Tier	Label	Meaning
0	No Risk	Ordinary text, no distress indicators
1	Mild Distress	Sadness/anxiety/stress language, no ideation
2	Moderate Risk	Passive suicidal ideation, no concrete plan
3	Severe Active Risk	Active intent, a method, and/or a timeline (e.g. "tonight")

There are two distinct code paths, and only one is usually active:

43. **Primary (ML) path** — `train_multi_class_baseline()` synthesizes 4-class labels for the training data via a keyword heuristic (e.g. `label==1` + a method/timeline keyword → tier 3), then fits a fresh `TfidfVectorizer` + multinomial `LogisticRegression` to predict the tier directly from text, saved to `models/multi_tier_classifier.pkl`. The docstring is explicit about *why* this exists: "Suicide Watch is binary" — there is no ground-truth 4-tier label anywhere in the source data, so the tiers used to train this model are themselves heuristically assigned, not human-annotated. **This should be stated plainly if asked** — it is a real limitation, not a hidden one.
44. **Fallback (rule) path** — used automatically if no trained multi-class `.pkl` exists (the default state of a freshly cloned repo, since this script is not part of the standard training sequence in Section 7): thresholds the *binary* model's risk probability — `<0.20`→Tier 0, `<0.50`→Tier 1, `<0.80`→Tier 2, `≥0.80` and a plan/method/timeline keyword present→Tier 3, `≥0.80` without one→Tier 2.

`sandbox.py`'s `get_multi_tier_classifier()` (an `lru_cached` singleton) tries to load the trained `.pkl` first and only falls back to the threshold rules if it's missing — so **which path is actually running depends on whether `multi_tier_classifier.py` has been run standalone**, which it is not by default. Check `models/multi_tier_classifier.pkl`'s existence before claiming which mode is live.

---

11. Out-of-Distribution Detection & Predictive Uncertainty

Both techniques are **transformer-only** (BERT/RoBERTa) — the TF-IDF baselines have no architecturally faithful equivalent (no raw pre-softmax logits in the same sense, no dropout layers), and are only computed when a caller explicitly opts in (`include_trust_signals=True`, used by the manual `/analyze` route and the multilingual route, but *not* the high-frequency live-monitor loop, for cost reasons).

- **Energy-based OOD detection** (Liu et al., 2020) — `TransformerClassifier.compute_energy()`: `E(x) = -T · logsumexp(logits / T)`. Lower energy = more in-distribution/confident; higher = more anomalous. The threshold is calibrated once at training time as the **95th percentile** of in-distribution validation-set energies (`OOD_PERCENTILE_THRESHOLD = 95`), saved to `models/{model}_ood.pkl`. At inference, an input whose energy exceeds that stored threshold is flagged `is_out_of_distribution: true`.
- **MC-Dropout predictive uncertainty** — `predict_with_uncertainty()`: runs **20** stochastic forward passes (`MC_DROPOUT_PASSES = 20`) with dropout layers explicitly kept in `.train()` mode while everything else stays in `.eval()`, averages the resulting class probabilities, and reports the **predictive entropy** of that average plus the standard deviation of the risk-class probability across passes. High entropy means the model's dropout subnetworks disagree with each other — a signal distinct from (and complementary to) calibrated confidence, which reflects only a single deterministic pass.

What/why/how for both techniques individually: [Section 26.12](#2612-energy-based-out-of-distribution-ood-detection) (OOD) and [26.13](#2613-mc-dropout-predictive-uncertainty) (MC-Dropout).

The frontend's `TrustSignals.jsx` component renders both (normalizing entropy by `ln(2)`, the max possible entropy for a binary distribution, into Low/Medium/High buckets at 0.33/0.66) — **but this component is not currently imported/rendered anywhere in the app** (confirmed by a project-wide import search). It is fully implemented and functionally correct, but effectively dead UI in the shipped app; the underlying `/analyze` response does carry `ood` and `uncertainty` fields correctly, so wiring it in is a small, low-risk follow-up rather than a rebuild.

---

12. Explainability (XAI) Suite

`backend/src/services/explainability.py` implements four independent attribution methods over the same `[word, score]` interface, where a positive score means "pushed the prediction toward risk" and negative means "pushed it toward non-risk". *(This section says what each method does in this codebase; [Section 26.7](#267-the-four-explainability-xai-methods) explains what each method fundamentally is and why it exists as a technique, e.g. why SHAP is grounded in game theory and why that makes it the slowest of the four.)*

45. **Baseline coefficient attribution** (`explain_baseline`) — for the linear models only: `score = TF-IDF value × learned coefficient`, computed directly from the model's weight vector (for

the calibrated SVM, averages the coefficients of the 3 cross-validated base estimators). Exact, not sampled.

46. **Leave-One-Out / LOO** (`explain_transformer_loo`) — removes one word at a time, re-runs inference, and measures the drop in risk-class probability. Simple, fast, robust — the default ("fast") explanation method for transformers.
47. **SHAP** (`explain_transformer_shap`) — wraps the HF `text-classification` pipeline in `shap.Explainer` (Partition explainer), `max_evals=500`. Grounded in Shapley values, but noticeably slower than LOO, especially on CPU.
48. **Integrated Gradients** (`explain_integrated_gradients`) — 20-step path integral of embedding-layer gradients from a zero baseline to the real input embedding; sub-token gradients are re-aggregated to word level by fuzzy substring matching against cleaned tokens. Falls back to LOO on any failure (e.g. an unsupported model architecture).
49. **LIME** (`explain_lime`) — `LimeTextExplainer`, 10 features, 100 perturbation samples (`LIME_NUM_FEATURES`/`LIME_NUM_SAMPLES`), needs a batched `predict_proba` callable (adapted per-classifier in `sandbox.py`'s `_predict_proba_batch`).

Multi-XAI Comparison Studio (`compare_explainers()`, surfaced as its own top-level frontend tab, `POST /api/explain-comparison`) runs LOO, Integrated Gradients, LIME, and SHAP on the same input and computes a full **pairwise Pearson correlation matrix** across the four attribution vectors (`compute_pearson_correlation`, a plain NumPy covariance/std implementation — no scipy dependency). High cross-method correlation is presented to the user as increased confidence that the highlighted words are genuinely load-bearing, not an artifact of one particular method.

What-If counterfactual tool (`what_if_swap()`, `POST /api/what-if`) — regex-swaps one target word/phrase for a replacement (word-boundary-safe, case-insensitive) and reports the before/after risk probability and whether the swap de-escalated the score. Useful for demonstrating sensitivity to specific lexical choices (e.g. swapping `"end my life"` → `"get help for my pain"`).

---

13. Governance & Trustworthiness Audits

13.1 Fairness audit — `services/fairness_auditor.py` + `services/fairness_cohort_data.py`

This is the area where an earlier draft of this document was most wrong, so it is worth stating precisely: this audit does not use demographic attributes (gender, age, socioeconomic status) at all — there is no such data in the Suicide Watch dataset, and the code makes no attempt to infer it. Instead it audits fairness across **three linguistic registers**:

- Youth Slang
- Formal Language
- Literal / Direct

`fairness_cohort_data.py` hand-authors **16 risk + 16 non-risk scenarios**, each written out in *all three registers expressing the same underlying situation* (e.g. the "academic failure" scenario appears as `"ngl i just failed my finals AGAIN..."`, `"I have failed my final examinations for`

`the second time...`", and "`I failed my exams again...`") — 32 scenarios × 3 registers = **96 total examples, 32 per cohort, class-balanced 16/16 within each cohort.** This parallel-content design is deliberate: it isolates *phrasing/register* as the only varying factor, so any measured gap is attributable to how something is said rather than what is being said.

For each cohort, the auditor computes:

- **Accuracy, Recall** (sensitivity to true risk), **Specificity** (correctly clearing true non-risk) — each with a **95% bootstrap confidence interval** (`n_bootstrap = 1000` resamples by default, percentile method, Efron & Tibshirani style; what/why/how of bootstrapping specifically: Section 26.10).
- A `meets_min_subgroup_size` flag (`FAIRNESS_MIN_SUBGROUP_SIZE = 30`) — a metric computed on fewer than 30 examples is explicitly labeled unreliable rather than reported as if trustworthy.
- **Cross-cohort fairness gaps**: for every metric and every cohort pair, a gap is only flagged if *both* cohorts clear the minimum size *and* their CIs don't overlap.

Verified results (Logistic Regression, from the landing page, cross-checked against the actual scenario counts in code): recall by register — Youth Slang 56.2%, Formal Language 68.8%, Literal/Direct 87.5%. But recall is computed only over the 16 true-risk examples per cohort — **below the audit's own 30-example minimum** — so this recall gap is *not* certified as statistically significant by the tool's own gating logic, even though the raw numbers look large. Accuracy, computed over the full 32-example cohort, does clear the threshold, and shows **zero statistically significant cross-cohort gaps at 95% CI**. This nuance (a visible-but-uncertified gap vs. a certified non-gap) is exactly the kind of result this auditor is designed to surface, and is a good example to walk through live if a professor asks how the fairness tooling actually works.

13.2 Construct validity audit — `services/construct_validity_auditor.py`

Directly implements the residualization protocol from **Dehghan & Ashrafi (2026)**, cited by name in the code. The question it answers: *is this model's apparent accuracy substantially explained by detecting generic negative sentiment, rather than suicide-specific risk language? (What "construct validity" and "confound" mean as general concepts, independent of this dataset: [Section 26.11](#2611-construct-validity--confound-residualization).)*

Method: build a hand-authored **generic negativity lexicon** (deliberately disjoint from crisis-specific words — no "suicide," "kill," "pills," "plan," etc. — just words like "sad," "tired," "hopeless," "burden," "stressed"), score each text's fraction of negativity-lexicon words, then linearly regress the model's predicted probability on that single negativity score. The **residual** (the part of the prediction the negativity confound *can't* explain) is then correlated with the true label:

- **High `negativity_r2`** → the model's output is substantially explained by generic negativity alone (a red flag).
- **High `residual_label_correlation`** → even after removing the negativity confound, the model still tracks the true label — evidence of genuine, construct-specific signal.

Verified live (`GET /api/construct-audit`, `n_samples=300`, matching the landing page's reported figures): Logistic Regression — negativity R^2 = **0.023**, residual–label correlation = **0.873**, raw model–label correlation = **0.885**. Reported figures for the other models: SVM 0.874, BERT 0.981, RoBERTa

0.975 residual correlation, with negativity R^2 for all four models in the 0.018–0.023 range. Interpretation: negative-sentiment overlap explains under 2.3% of variance in any model's output, and residual correlation stays high (0.87–0.98) — i.e. these classifiers are not simply generic sadness detectors in disguise.

13.3 Robustness testing — `services/robustness.py`

Two synthetic perturbations, applied to the full test split for every trained model, then re-evaluated:

- **Typo injection** (`inject_typos`, rate 0.15 per eligible word): randomly swaps adjacent characters, drops a character, or doubles a character, only on words longer than 3 characters.
- **Distracting text** (`add_distracting_text`): appends one of five deliberately unrelated, mildly positive sentences (e.g. *"Anyway, I'm going to watch a comedy movie now and eat pizza."*) to the end of the input, to test whether an irrelevant positive coda can drag down a correct risk prediction.

Results are saved per model to `models/robustness_metrics.pkl` and surfaced in the Analytics tab. BERT's reported drop under typos (−1.83 F1 points) is notably larger than under the distracting-text perturbation (−0.31 points) — i.e. the model is more sensitive to surface-level lexical noise than to an irrelevant but grammatical coda, which is a defensible, explainable pattern to discuss if asked why the two perturbations differ.

13.4 Model drift — `services/drift_detector.py`

(What PSI is and why it's the right tool for drift specifically, as opposed to just re-checking accuracy: [Section 26.9](#269-population-stability-index-psi).)

Population Stability Index (PSI) between two probability distributions (5 bins over `[0, 1]`): `PSI = Σ (act% − exp%) × ln(act% / exp%)`. Thresholds actually implemented in code: **`< 0.10` = stable (green)**, **`0.10 ≤ PSI < 0.20` = moderate/warning (amber)**, **`PSI ≥ 0.20` = critical (red)**. *(An earlier draft of this document stated the critical threshold as 0.25 — the code uses 0.20; use the code's number.)* In the running app, "baseline" is the distribution of manually-analyzed (Sandbox-tab) predictions and "stream" is the distribution of Live-Monitor predictions — so PSI here measures drift between *how a human operator uses the tool* and *what the simulated live feed produces*, not drift against a frozen training-time reference distribution. If fewer than 5 examples exist in either bucket, or on any DB error, the endpoint returns a fixed placeholder/default response rather than a broken chart — worth knowing so a freshly-seeded database doesn't look like a bug.

---

14. Clinical Decision-Support Layer

A set of services that turn a raw model prediction into something an operator can act on. **Everything in this layer that looks like it takes a real-world action is explicitly simulated** — this is a deliberate, stated safety boundary, not an oversight.

- **Emotion analyzer** (`emotion_analyzer.py`) — 6 lexical categories: sadness, anger, fear, hopelessness, anxiety, joy (word + bigram matching, normalized to sum to 1). Optionally backed

by a real Hugging Face emotion-classification pipeline (`bhadresh-savani/distilbert-base-uncased-emotion`) if `use_transformer=True` is passed at construction, with automatic fallback to the lexical method on any load/inference failure — but the app's `sandbox.py` singleton constructs `EmotionAnalyzer()` with default arguments, so **the lexical path is what actually runs in the shipped app**, not the transformer path. *(Correction vs. an earlier draft: there are 6 categories here, not 8 — "disgust," "surprise," and "trust" as separate dimensions do not exist in this code.)*

- **Cognitive distortion analyzer** (`cognitive_distortion_analyzer.py`) — 10 Beck/Burns CBT distortion categories (All-or-Nothing Thinking, Overgeneralization, Mental Filter, Disqualifying the Positive, Jumping to Conclusions, Magnification/Catastrophizing, Emotional Reasoning, Should Statements, Labeling, Personalization), each a list of case-insensitive regex patterns. Scored per-category as match-count normalized by word count; `get_dominant_distortions()` returns the top 3 categories clearing a minimum score of 0.03.
- **Clinical response helper** (`clinical_helper.py`) — 4 hand-written draft-response templates, one per risk tier, each following explicit safety principles (validate without diagnosing; never suggest medication/diagnosis; always provide crisis contact info at tier ≥2; escalate to 988 at tier 3). If a dominant cognitive distortion was detected, a gentle, **non-diagnostic reflective sentence** is woven in (deliberately avoids naming the distortion to the person experiencing it, since that would read as judgmental).
- **Anonymizer** — see Section 5.
- **Semantic case search** (`semantic_search.py`) — retrieves similar historical cases via **TF-IDF cosine similarity** against a small seed set (5 cases in `db.py`'s `SEED_CASES`, extensible via `insert_case`). Despite the "semantic" naming, this is lexical/keyword overlap, not embedding-based semantic similarity — a real limitation worth naming if asked how "semantic" the search actually is, and a natural next step (e.g. swapping in real sentence embeddings) if there's time to extend the project.
- **Report generator** (`report_generator.py`) — produces a self-contained, print-styled HTML clinical report (tier badge, anonymized post, empathy draft response, explicit non-diagnostic disclaimer) — pure Python f-string templating, no external templating engine.
- **Clinical Copilot** (`clinical_copilot.py`) — `generate_compliance_audit()` computes a deterministic HIPAA-style "compliance hash" (`SHA-256` over the raw text + tier + probability + timestamp, truncated to 16 hex chars) as a tamper-evident audit token, plus a triage priority/protocol/hotline/action-item bundle keyed off tier and probability. `dispatch_safety_protocol()` **is explicitly and unambiguously simulated** — every response is prefixed `[SIMULATED]` and the docstring states in plain language that it "does NOT place a real call, contact a real crisis line, or notify any real person." There is no integration with 988, a supervisor queue, or any external service.

---

15. The Gemini-Backed RAG Narrative (new this session)

`services/rag_copilot.py`'s `ClinicalRAGCopilotEngine.query_rag_knowledge()` retrieves matches against a small hardcoded knowledge base (3 DSM-5 diagnostic-criteria entries with

keyword lists, and a 3-level C-SSRS protocol table) and deterministically selects a triage protocol from the input's probability and a handful of trigger keywords — **all of this logic is unchanged, code-only, and does not involve any LLM.**

What was added this session is a **narrative layer on top of that already-decided output**: `services/gemini_client.py` wraps the `google-genai` SDK (model `gemini-flash-latest`, thinking explicitly set to `MINIMAL` since this task needs a short direct answer, not extended reasoning) and `rag_copilot.py` now asks it to write a 2–3 sentence clinician-facing rationale — but the system instruction explicitly forbids the model from inventing facts or **overriding the already-computed C-SSRS tier**, and the input text is passed through `mask_pii()` before it ever leaves the process. The response payload gains a `narrative_source: "gemini" | "template"` field so the UI never overstates what generated the text, and `generate_text()` swallows every possible failure (missing key, network error, quota, unsupported request shape) and returns `None`, in which case the original deterministic one-line template is used instead — **the feature is additive and fails safe; nothing else in the app depends on it.**

This design directly reflects the literature-review finding (Section 3) that LLMs are unreliable when trusted to reason end-to-end about clinical risk: here, the LLM is scoped to *paraphrasing a decision*, never to *making* one. *(What RAG means in general, and exactly which half of that definition this feature does vs. doesn't implement: [Section 26.16](#2616-retrieval-augmented-generation-rag).)*

---

16. Multilingual Analysis

`services/translation.py` + `POST /api/multilingual-analyze`: detects a small set of languages (es/fr/de/hi/zh/it/pt/ru/ar/ja/ko) via keyword-indicator scoring, translates to English via (in order) an exact-match lookup in a small hardcoded phrase dictionary, then `deep_translator`'s `GoogleTranslator` if installed, then a crude word-for-word substitution as a last resort — then runs the full standard analysis pipeline on the English translation and **projects the resulting word-attribution scores back onto the original-language tokens proportionally by position** (`align_attributions_to_source`). The frontend's language-routing heuristic that decides *whether* to call this endpoint at all (`AnalysisContext.jsx`) is itself a blunt regex (any non-ASCII character, or a short hardcoded list of Spanish/French/German function-word fragments) — both the translation quality and the routing decision are explicitly heuristic layers, not a real translation model, and should be described as such rather than as "multilingual NLP."

---

17. Live Monitor & Temporal Trend Detection

Two related but distinct features that are easy to conflate. *(What/why/how of the underlying techniques: change-point detection — [26.14](#2614-change-point-detection-binary-segmentation); linear trend slope — [26.15](#2615-linear-trend-detection); WebSockets — [26.17](#2617-websockets-live-monitor).)*

- **Live Monitor** (`monitor_manager.py`, `feed_simulator.py`, `trend_analyzer.py`, WebSocket at `/api/ws/monitor`) — a background task ticks every 6 seconds, pulling the next post from one of **5 synthetic users, each with a fixed 6-post storyline** (escalating, flat/benign, flat/moderate, de-escalating, flat/high-risk), runs it through the full analysis pipeline, persists it to SQLite (`source='monitor'`), and broadcasts it to every connected WebSocket client. Per-user trend detection is **real, computed live from actual logged history**: `compute_trend()` fits an OLS linear slope through a user's probability history and classifies it Escalating/Stable/De-escalating (thresholds ±0.05), and `detect_change_point()` runs `ruptures` Binary Segmentation (`jump=1, min_size=1`, chosen specifically because the default `jump=5` throws on the short histories — as few as 4–6 posts — this app deals with) to flag a single abrupt shift, gated by both a minimum history length (4) and a minimum magnitude (0.15) so it doesn't over-claim a "shift" from small-sample noise.
- **Temporal Trajectory tab** (`api/v1/endpoints/temporal.py`) — by contrast, this scores a **fixed, hardcoded 4-post demo timeline** (dated 2026-07-01 through 2026-07-15, a scripted escalation from "normal day" to "everything is hopeless") through whichever model is currently selected. **It is not connected to real logged user history at all** — this is explicitly called out as a limitation on the landing page, and is worth stating proactively: the Live Monitor's trend engine is the real, general-purpose version of this idea; the Temporal tab is a fixed illustrative demo.

---

18. Backend Architecture

- **Framework:** FastAPI, ASGI, run via `uvicorn`.
- **Entry point:** `backend/src/main.py` — builds the `FastAPI` app, registers CORS middleware, a structured-logging HTTP middleware (per-request correlation ID, method/path/status/latency), a catch-all exception handler (logs full tracebacks server-side, returns a generic 500 to the client so internals never leak), mounts the API under `/api`, and loads `.env` (via `python-dotenv`) for `GOOGLE_API_KEY` at startup.
- **Routing layers:** `api/routes.py` → `api/v1/router.py` → seven per-feature routers under `api/v1/endpoints/` (`health`, `sandbox`, `cases`, `analytics`, `temporal`, `monitor`, `copilot`), each independently importable and independently prefixed by the shared `sys.path` bootstrap at the top of nearly every backend file (a consequence of running the backend as a plain script rather than an installed package).
- **Blocking work off the event loop:** every route that touches a model, the filesystem, or SQLite wraps its synchronous logic in `asyncio.to_thread(...)`, keeping the async event loop free — the one documented exception is `monitor_service.start()`, which must run directly on the event loop because it calls `asyncio.create_task()` (calling that via `to_thread` previously caused a "no running event loop" bug, per an explanatory code comment). *(What/why/how of this pattern in general: [Section 26.19](#2619-asynciotothread-backend-concurrency).)*
- **Model loading/caching:** `sandbox.py` uses `functools.lru_cache` singletons (`get_baseline_classifier`, `get_transformer_classifier`, `get_multi_tier_classifier`, `get_emotion_analyzer`, `get_calibration`, `get_ood_calibration`) so each model/artifact is loaded from disk once per process, not per request.

- **Persistence:** SQLite (`data/app.db`, via `db.py`, stdlib `sqlite3`) — two tables: `cases` (seeded historical case-resolution pairs for semantic search) and `analyses` (every manual and monitor-sourced prediction, with a lightweight migration path for the `user_id` column on pre-existing databases).
- **Logging:** `logging_config.py` provides `get_logger(name)`, used consistently across every service and route for structured, per-module logging.

---

19. Full API Endpoint Catalog

All prefixed with `/api`.

Method	Path	Purpose
GET	`/status`, `/health`	Liveness checks
POST	`/analyze`	Full single-post analysis pipeline (prediction, tier, explanation, emotion, distortions, draft response, optional OOD/uncertainty)
POST	`/what-if`	Counterfactual word-swap re-analysis
POST	`/explain-comparison`	4-explainer Multi-XAI comparison + correlation matrix
POST	`/multilingual-analyze`	Translate + analyze + back-project attributions
GET	`/cases`	List seeded/stored historical cases
POST	`/search`	TF-IDF similarity search over cases
POST	`/report`	Generate a print-styled HTML clinical report
GET	`/fairness`	Linguistic-register fairness audit
GET	`/construct-audit`	Construct-validity (negativity-confound) audit
GET	`/metrics`, `/model-metrics`	Test-set accuracy/precision/recall/F1

		per model
GET	`/robustness`	Typo/distraction robustness metrics per model
GET	`/drift-metrics`	PSI + histogram + emotion shift, baseline vs. live stream
GET	`/temporal`	Fixed 4-post demo timeline scored by the selected model
POST	`/monitor/start`, `/monitor/stop`	Start/stop the simulated live feed
GET	`/monitor/status`	Current monitor running state
GET	`/monitor/users`	Per-synthetic-user trend/change-point snapshot
WS	`/ws/monitor`	Live event stream (history on connect, then real-time ticks)
POST	`/copilot/rag-query`	DSM-5/C-SSRS grounded retrieval + Gemini-or-template narrative
POST	`/copilot/audit`	HIPAA-style compliance audit + triage bundle
POST	`/copilot/dispatch-protocol`	**Simulated** safety-protocol dispatch

---

20. Frontend Architecture

- **Stack:** React 19, Vite 8, `lucide-react` icons, `oxlint` for linting. No CSS framework — a hand-built design-token system in `App.css` (`--bg-surface-*`, `--text-*`, `--color-brand/-success/-warning/-danger/-info`, all redefined per theme under `[data-theme='light']`). No router — `react-router` is not a dependency.
- **State management** *(what/why/how of Context itself, vs. Redux/prop-drilling: [Section 26.18](#2618-react-context-api-frontend-state-management))*: five React Context providers, composed in `App.jsx`: `ThemeProvider` (persists to `localStorage`, sets a `data-theme` attribute the CSS keys off), `NotificationProvider` (toast queue), `AppProvider` (active tab, selected model, modal open-states), `AnalysisProvider` (all Sandbox/What-If/Cases/Fairness/Temporal/Analytics state and API calls), `MonitorProvider` (live-feed state + WebSocket lifecycle). Every provider has a matching one-line `useX()` hook under `hooks/` that throws if called outside its provider.

- **Navigation:** a collapsible left `Sidebar` (grouped into 5 categories: Core Assessment, Governance & Audits, Simulation & Search, Bias & Trajectory, Real-Time Monitoring) plus a `Header` showing the active tab's title, model selector, command palette shortcut, Copilot button, and theme toggle. A Ctrl/Cmd+K `CommandPalette` duplicates quick navigation/model-switch/theme-toggle actions as a fuzzy-searchable overlay — note it does **not** currently list the Multi-XAI Studio or Model Drift & PSI tabs, even though both exist and are reachable from the sidebar.
- **Tabs (9 total):** Diagnostic Assessment (Sandbox), Multi-XAI Studio, Clinical Analytics, Model Drift & PSI, What-If Perturbation, Historical Retrieval, Demographic Audit, Temporal Trajectory, Live Stream Monitor — all lazily code-split (`React.lazy`) and kept mounted-but-hidden (`display: none` rather than unmount) when inactive, so switching tabs never re-fetches or loses in-progress input.
- **Landing page** (`components/landing/LandingPage.jsx`) — a fully separate, self-styled marketing/report page shown before the dashboard (toggled by a local boolean in `App.jsx`, not a route — reloading the page always returns to it). Static, no API calls. Carries the byline, dataset/evaluation/fairness/construct-validity summary tables, and an illustrative (non-live) SHAP token simulator. **This is the most convenient single page to present from** if you want the headline numbers on screen without navigating the live dashboard.
- **Known-orphaned components** — `CognitiveDistortions.jsx`, `AnonymizerDiff.jsx`, and `TrustSignals.jsx` are fully implemented, styled, and shaped to match real backend response fields, but are not currently imported anywhere in the live app (confirmed by project-wide import search). They represent implemented-but-unwired functionality, not missing functionality — a natural, low-risk thing to point to if asked "what would you do with another week."

---

21. Complete Directory Structure

```
project-3/

├── .env / .env.example          # GOOGLE_API_KEY (gitignored; example is the
tracked template)

├── README.md                    # Setup instructions

├── requirements.txt             # Backend + ML dependencies

├── run.bat / run.sh             # One-command launcher (creates
venv/node_modules if missing, runs both servers)

├── PROJECT_PLAYBOOK.md          # This document

│
```

```
├── literature_review/           # 16-paper structured literature review (.md +
.docx) + 14 source PDFs

│

├── ai_model/src/

│   ├── baseline_models.py       # TF-IDF + Logistic Regression / calibrated SVM

│   ├── transformer_models.py    # BERT/RoBERTa fine-tuning, EMA, OOD, MC-
Dropout

│   ├── multi_tier_classifier.py # 4-tier escalation (ML path + rule-fallback
path)

│   └── calibration.py           # Temperature scaling + ECE

│

├── backend/src/

│   ├── main.py                  # FastAPI app, CORS, logging middleware, .env
loading

│   ├── db.py                    # SQLite schema (cases, analyses) + queries

│   ├── logging_config.py        # Shared structured logger factory

│   ├── config/settings.py       # Every tunable constant in the system,
centralized

│   ├── api/

│   │   ├── routes.py            # Mounts the v1 router

│   │   └── v1/

│   │       ├── router.py

│   │       └── endpoints/       # health, sandbox, cases, analytics, temporal,
monitor, copilot

│   ├── schemas/                 # Pydantic request/response models (analyze,
cases, monitor, copilot)
```

```
│   ├── services/                # All business logic — 20 modules, see Sections
12–17
│   └── utils/preprocessing.py
│
├── data/
│   ├── Suicide_Detection.csv    # Raw Kaggle dataset (not committed by default)
│   ├── app.db                   # SQLite runtime database
│   └── processed/               # train.csv / val.csv / test.csv + 2 EDA plots
│
├── models/                      # All trained artifacts: vectorizer, 4
classifiers,
│                                 # per-model metrics/calibration/OOD .pkl
files,
│                                 # bert_model/ and roberta-base/ checkpoint
directories
│
├── frontend/
│   ├── index.html, main.jsx, App.jsx, App.css
│   ├── src/components/
│   │   ├── layout/      Sidebar, Header, CommandPalette
│   │   ├── sandbox/      SandboxTab, WordImportance, MultilingualHeatmap,
│   │   │                 MultiXAIComparisonStudio, CognitiveDistortions*,
│   │   │                 AnonymizerDiff*, TrustSignals*   (* = implemented, not
wired in)
│   │   ├── analytics/    AnalyticsTab, DriftDashboard
│   │   ├── copilot/      ClinicalCopilotModal
```

```
│   │   ├── cases/ fairness/ temporal/ whatif/ monitor/   (one tab component
each)
│   │   └── landing/      LandingPage
│   ├── src/context/      Theme, App, Analysis, Monitor, Notification
│   ├── src/hooks/        one-line useX() wrapper per context
│   └── src/services/apiClient.js   # fetch-based API client (no axios)
│
└── tests/                        # 8 pytest files, 70 passing test functions
(see Section 23)
```

---

22. How to Run & Reproduce

```
# 1. Python environment
python -m venv venv && venv\Scripts\activate        # (or source
venv/bin/activate on macOS/Linux)
pip install -r requirements.txt


# 2. Frontend dependencies
cd frontend && npm install && cd ..


# 3. Dataset — download Suicide_Detection.csv from Kaggle, place at
data/Suicide_Detection.csv


# 4. Preprocess
python backend/src/utils/preprocessing.py


# 5. Train baselines (LR + calibrated SVM + TF-IDF vectorizer)
python ai_model/src/baseline_models.py
```

```
# 6. Fine-tune transformers (repeat for --model roberta; add --quick for a fast
smoke test)

python ai_model/src/transformer_models.py --model bert


# 7. (Optional, not part of the core sequence) robustness / construct-validity /
multi-tier audits

python -m services.robustness                 # from backend/src, or adjust
sys.path

python -m services.construct_validity_auditor

python ai_model/src/multi_tier_classifier.py


# 8. Run the backend

cd backend/src && uvicorn main:app --reload --port 8000


# 9. Run the frontend (separate terminal)

cd frontend && npm run dev     # http://localhost:5173


# 10. Run the test suite

pytest tests/ -v
```

`run.bat` / `run.sh` automate steps 1–2 and 8–9 (venv/node_modules creation if missing, then both servers) — the fastest path to a live demo if models are already trained.

Gemini narrative (optional): copy `.env.example` to `.env` and set `GOOGLE_API_KEY`. Everything else in the app works identically with or without it — the RAG copilot narrative just falls back to its deterministic template.

---

23. Test Suite

8 files, 70 passing test functions (verified by running `pytest tests/ -v` live this session — all 70 passed, ~134s wall clock, model-loading dominated). Correcting an earlier draft's fabricated file list — the real files are:

File	Focus

`test_advanced_features.py`	Anonymizer, multi-tier rules, emotion lexicon, clinical-helper drafts, semantic search, what-if, cognitive distortions, construct-validity audit
`test_api_routes.py`	Full FastAPI route coverage — `/analyze` across all 4 models, anonymization, LIME/SHAP explanation gating, what-if, search, robustness/construct/fairness/metrics endpoints, temporal trend, report generation
`test_drift_detector.py`	PSI calculation + drift-metrics response shape
`test_monitor_and_db.py`	DB init/idempotency/inserts, monitor start/stop lifecycle, WebSocket history replay, trend/change-point classification
`test_pipeline.py`	Text cleaning, config paths, robustness perturbation functions, direct anonymizer tests
`test_rag_copilot.py`	RAG knowledge retrieval + C-SSRS protocol selection, **and** (added this session) the Gemini-narrative fallback/success paths, mocked to stay network-free
`test_translation_pipeline.py`	Language detection, translation, token alignment
`test_xai_comparison.py`	Pearson correlation helper

No frontend test suite exists (no Vitest/Jest configuration in `frontend/`) — if asked about frontend test coverage, the honest answer is zero automated tests, manual verification only.

---

24. Known Limitations (say these before your professor finds them)

Stating these proactively is stronger than waiting to be asked — it demonstrates you understand the system's actual boundaries, not just its features.

50. **The 4-tier risk labels are heuristically synthesized, not human-annotated** — Suicide Watch is a binary-labeled dataset; the multi-tier model (when trained) learns from keyword-assigned pseudo-labels, and the fallback path is explicit threshold rules. Neither is clinically validated.
51. **Fairness cohorts are small.** 16 true-risk examples per linguistic register is below the audit's own 30-example significance threshold — the visible recall gap (56.2% vs. 87.5%) is real in the data but not statistically certified by the tool's own standard. More scenarios per cohort would be needed to make that claim rigorously.
52. **The Temporal Trajectory tab scores a fixed, hardcoded 4-post demo timeline**, not real per-user logged history — that's what the Live Monitor's trend engine is for.

53. **"Semantic" case search is TF-IDF/lexical, not embedding-based.** It will miss conceptually similar cases phrased with different vocabulary.
54. **Multilingual translation is a small hardcoded dictionary + optional third-party fallback + crude word substitution**, not a translation model — quality degrades quickly outside the handful of pre-seeded example phrases.
55. **All "dispatch"/"action" behavior in the Clinical Copilot is simulated.** No real integration exists with 988, a supervisor queue, or any external system — this is by design, not a missing feature, given the safety implications of a research prototype claiming to place real crisis calls.
56. **OOD detection and MC-Dropout uncertainty only exist for BERT/RoBERTa**, not the TF-IDF baselines.
57. **`TrustSignals`, `CognitiveDistortions`, and `AnonymizerDiff` frontend components are implemented but not currently rendered anywhere** in the live app.
58. **No authentication, authorization, or rate limiting on any API endpoint**, and CORS is configured permissively (`allow_origins=["*"]` combined with `allow_credentials=True`) — acceptable for a local research demo, not for any real deployment.
59. **PII anonymization is regex-based**, not a trained NER model — it will miss names/identifiers that don't match its two supported sentence patterns.

---

25. Glossary

Term	Meaning
XAI	Explainable AI — methods that attribute a model's output to its inputs
SHAP	SHapley Additive exPlanations — game-theoretic feature attribution
LIME	Local Interpretable Model-agnostic Explanations — local linear surrogate
IG	Integrated Gradients — path-integral gradient attribution
LOO	Leave-One-Out — attribution via single-feature ablation
ECE	Expected Calibration Error — gap between confidence and accuracy
PSI	Population Stability Index — distributional drift metric
OOD	Out-of-Distribution — input unlike anything seen in training
EMA	Exponential Moving Average (of model weights

	during training)
C-SSRS	Columbia-Suicide Severity Rating Scale — clinical triage protocol
DSM-5	Diagnostic and Statistical Manual of Mental Disorders, 5th ed.
RAG	Retrieval-Augmented Generation
CI	Confidence Interval
TF-IDF	Term Frequency–Inverse Document Frequency (bag-of-words weighting)
CBT	Cognitive Behavioral Therapy (source of the Beck/Burns distortion taxonomy)

---

26. Core Concepts Explained — What, Why, How

The Glossary above is a lookup table. This section is the version you actually study from — every non-trivial technique used anywhere in the system, explained as **What it is → Why this project uses it → How it actually works**, in the order you'd naturally hit them walking through the pipeline. If a professor stops you mid-sentence and asks "wait, what is that," the answer is here.

26.1 TF-IDF (Term Frequency–Inverse Document Frequency)

- **What it is:** a way to turn a piece of text into a vector of numbers so a classical ML model (which only understands numbers) can use it. Each dimension of the vector corresponds to one word (or, here, one word/two-word phrase) from a fixed vocabulary.
- **Why used here:** it's the input representation for the two baseline models (Logistic Regression, SVM). The project needs at least one model family whose predictions can be explained *exactly* — TF-IDF + a linear model is the simplest representation where "why did it predict risk" has a literal, closed-form answer (see 26.7).
- **How it works:** for a word *w* in document *d*, `TF(w,d)` = how often *w* appears in *d*; `IDF(w)` = `log(N / (number of documents containing w))`, so words that appear in *almost every* document (like "the") get pushed toward zero, and words that are rare-but-present in a document get a high weight. The score is `TF × IDF`. This codebase's vectorizer keeps the **top 5,000** highest-scoring unigrams+bigrams (`ngram_range=(1,2)`) across the training set as its fixed vocabulary — so `"end"` and `"end my life"` can each be their own dimension.

26.2 Logistic Regression & Linear SVM

- **What they are:** two classic linear classifiers. Both learn one weight per input dimension (here, one weight per TF-IDF vocabulary word) and combine them with a dot product; Logistic

Regression squashes that sum through a sigmoid to get a probability, LinearSVC instead finds the hyperplane that maximizes the margin between the two classes.

- **Why used here:** cheap to train (seconds, CPU-only), and — critically — **fully transparent**: the sign and magnitude of a word's learned weight *is* its contribution to the prediction, no approximation needed.
- **How it works:** during training, both algorithms iteratively adjust the per-word weights to reduce classification error on the training set (Logistic Regression minimizes log-loss; LinearSVC maximizes margin / minimizes hinge loss). Neither can natively output a calibrated probability for a "confidence" that's directly comparable to the transformers, which is why the SVM is wrapped in `CalibratedClassifierCV` (see 26.4) and why *all four* models later go through the project's own temperature-scaling layer (26.8).

26.3 Transformers, BERT & RoBERTa

- **What they are:** deep neural networks that read a whole sentence at once through a mechanism called **self-attention**, letting every word's representation be informed by every other word in the sentence (not just its immediate neighbors, the way older RNN-style models worked). BERT and RoBERTa are two specific pretrained transformer models — both were first trained on huge amounts of unlabeled text (masked-word prediction) before this project ever touched them.
- **Why used here:** context. TF-IDF has no concept of word order or negation — `"I don't want to die"` and `"I want to die"` can look almost identical to a bag-of-words model. A transformer's attention mechanism can, in principle, learn that `"don't"` flips the meaning of everything after it. That's the ~6-point F1 gap between the linear models and BERT/RoBERTa in Section 9.
- **How it works (at fine-tuning level, which is what this project actually does):** the project does **not** train a transformer from scratch — that would need far more data and compute than a 15,000-post dataset provides. Instead it downloads the already-pretrained `bert-base-uncased`/`roberta-base` weights from Hugging Face, replaces the final layer with a fresh 2-class classification head, and **fine-tunes** — i.e. continues training, at a very small learning rate (`2e-5`), for a few epochs — so the model's general language understanding gets specialized toward "does this text express suicide risk." See Section 6.3 for the exact hyperparameters, and 26.5–26.6 below for why fine-tuning needed extra stabilization machinery beyond a plain training loop.
- **BERT vs. RoBERTa, briefly:** RoBERTa is BERT's architecture with a revised pretraining recipe (more data, longer training, dynamic masking, no next-sentence-prediction objective) — the project fine-tunes both because they end up nearly tied here (97.68 vs. 97.65 F1), which is itself a useful result to discuss: on this task, the pretraining-recipe differences that separate the two models on general NLP benchmarks mostly wash out once both are fine-tuned on the same 10,500-post domain-specific dataset.

26.4 Why the SVM needs `CalibratedClassifierCV`

- **What it is:** a scikit-learn wrapper that fits several copies of a base classifier on different folds of the training data and uses their held-out predictions to fit a small calibration model (Platt/sigmoid scaling) mapping the base classifier's raw decision score to a probability.

- **Why used here:** `LinearSVC` (unlike Logistic Regression) has no built-in concept of a probability — it only outputs a signed distance from the decision boundary. But this project's entire architecture (multi-tier thresholds, calibration, OOD, every UI element that shows a "risk %") assumes every model exposes `.predict_proba()`. The wrapper exists purely to give the SVM that interface.
- **How it works:** `CalibratedClassifierCV(base_svm, cv=3)` trains 3 SVMs on 3 different train/held-out folds, and for each fold fits a sigmoid mapping the SVM's decision score → probability using the held-out predictions (never training and calibrating on the same rows). The final `predict_proba` averages across the 3 fold-specific calibrators.

26.5 Fine-Tuning Stabilizers: Cosine Warmup, Mixed Precision, Gradient Clipping

These three are grouped because they answer the same underlying question — *fine-tuning a large pretrained model on a small dataset is numerically fragile; what stops it from diverging or wasting compute?*

- **Cosine schedule with warmup — what/why/how:** the learning rate doesn't jump straight to `2e-5` — it ramps up linearly for the first 10% of training steps (`WARMUP_RATIO = 0.1`), then decays smoothly along a cosine curve to ~0 by the end. **Why:** starting at full learning rate on a pretrained model's weights risks a large, destabilizing update before the newly-initialized classification head has learned anything sensible; warmup avoids that, and the cosine decay lets the model settle into a minimum rather than oscillating around it in the final steps.
- **Mixed precision (AMP) — what/why/how:** `torch.amp.GradScaler` runs most of the forward/backward pass in 16-bit floating point instead of 32-bit, only using 32-bit where 16-bit would lose too much precision (the "scaler" rescales gradients to avoid them underflowing to zero in 16-bit). **Why:** roughly halves GPU memory use and meaningfully speeds up training on a CUDA GPU, with negligible accuracy cost — but it's **only enabled when CUDA is available** (`USE_MIXED_PRECISION = True` combined with a CUDA check), since it provides no benefit on CPU.
- **Gradient clipping — what/why/how:** after computing gradients each step, their global norm is capped at `1.0` (`clip_grad_norm_`) before the optimizer applies them. **Why:** an occasional batch can produce an unusually large gradient (e.g. from a confusing or mislabeled example); clipping prevents one bad batch from taking a destructively large step and derailing training.

26.6 Exponential Moving Average (EMA) of Weights

- **What it is:** instead of using the model's weights from the very last training step, maintain a separate "shadow" copy that is a slow-moving weighted average of the weights across *many* recent steps.
- **Why used here:** the last few steps of fine-tuning can be noisy — the raw final-step weights might happen to reflect an unlucky batch. Averaging smooths that noise out, generally producing a more stable, better-generalizing checkpoint. The code goes further than just applying this at the end: it evaluates *and saves* the EMA-averaged weights throughout training, not the raw ones.
- **How it works:** after every optimizer step, each parameter's shadow value is updated as `shadow = decay × shadow + (1 − decay) × current_weight`, with `decay = 0.999` — meaning the

shadow changes very slowly and effectively remembers ~1,000 recent steps. Because you can't evaluate two sets of weights in the same model simultaneously, `evaluate_with_ema()` temporarily swaps the EMA weights in, evaluates, then swaps the *live* training weights back so training can continue uninterrupted — see `ai_model/src/transformer_models.py`.

26.7 The Four Explainability (XAI) Methods

All four answer the same question — *which words pushed this specific prediction toward "risk"?* — but by fundamentally different mechanisms, which is exactly why comparing them (the Multi-XAI Studio) is informative rather than redundant.

- **Baseline coefficient attribution — what/why/how:** *only* possible for the linear models. **What:** for a given input, each word's contribution is literally `(that word's TF-IDF value in this document) × (that word's learned model coefficient)`. **Why:** this is the one method in the whole suite that is *exact*, not estimated — there's no approximation to critique. **How:** direct arithmetic on already-known numbers; effectively free to compute.
- **Leave-One-Out (LOO) — what/why/how: What:** remove one word from the sentence, re-run the model, see how much the risk probability changed. **Why:** it's the simplest possible causal test ("does the prediction depend on this word?") and needs nothing but the ability to call the model repeatedly — no gradients, no sampling theory — which makes it fast and robust enough to be the project's *default* transformer explanation method. **How:** for an *n*-word sentence, run the model *n+1* times (once whole, once per word removed) and record each probability drop.
- **SHAP (SHapley Additive exPlanations) — what/why/how: What:** a game-theory-grounded method that treats each word as a "player" contributing to a "payout" (the prediction), and computes each word's *fair share* of that payout — its Shapley value — by (conceptually) averaging its marginal contribution across every possible subset of the other words. **Why:** Shapley values are the unique attribution method satisfying a specific set of fairness axioms from cooperative game theory (efficiency, symmetry, additivity), which is why SHAP is the most theoretically well-grounded of the four — at the cost of being the slowest, since exhaustively trying every subset is exponential, so this project's `shap.Explainer` uses a Partition explainer approximation capped at `max_evals=500` samples rather than the true exponential computation. **How:** wraps the Hugging Face classification pipeline and samples enough subsets/permutations to approximate each token's Shapley value.
- **Integrated Gradients (IG) — what/why/how: What:** a gradient-based method specific to differentiable models (i.e. neural networks, not the linear baselines). **Why:** raw gradients at a single point can be misleading/noisy for deep networks (their "saturation" problem); IG instead integrates the gradient along an entire straight-line path from a neutral baseline input (here, all-zero embeddings) to the real input, which satisfies a completeness axiom the linear baseline method also satisfies (the attributions sum exactly to the difference between the baseline's and the real input's prediction). **How:** the code takes 20 interpolation steps between the zero-baseline and the real embeddings, computes the gradient of the risk-class logit at each step, averages them, and multiplies by `(input - baseline)` — then aggregates sub-word gradients back to whole words by fuzzy string matching.
- **LIME (Local Interpretable Model-agnostic Explanations) — what/why/how: What:** rather than opening up the model at all, LIME repeatedly perturbs the input (masking out random words),

observes how the model's output changes across those perturbations, and fits a simple, interpretable *local* model (here, effectively a linear one) to approximate the real model's behavior *just in the neighborhood of this one input*. **Why:** it's completely model-agnostic — it would work identically on a model this project doesn't even have code-level access to — which makes it a useful independent cross-check against the gradient/game-theory-based methods above. **How:** `LimeTextExplainer` generates 100 perturbed variants (`LIME_NUM_SAMPLES`), reweights them by similarity to the original, and fits a sparse linear model over the top 10 features (`LIME_NUM_FEATURES`).

- **Why compare all four via Pearson correlation:** each method has a different failure mode (SHAP's sampling approximation, IG's gradient-saturation risk on deep layers, LIME's sensitivity to the perturbation sampling, LOO's blindness to interaction effects between words). If independently-derived methods agree on which words mattered, that convergence is much stronger evidence than any single method's output — this is exactly what the Explainer Convergence Matrix (`compute_pearson_correlation`) is for: a plain covariance-over-standard-deviations Pearson coefficient between every pair of the four attribution vectors, so a reviewer can see numerically, not just visually, how much the methods agree.

26.8 Temperature Scaling & Expected Calibration Error (ECE)

- **What "calibration" means here:** a model can be *accurate* (mostly right) while being *miscalibrated* (its stated confidence doesn't match its actual correctness rate) — e.g. a model that says "85% risk" but is only actually right 60% of the time when it says that. Calibration fixes the *second* problem without touching the *first* (it never changes which class is predicted, only how confidently).
- **Why used here:** several downstream features (the multi-tier probability thresholds, the "confidence" a clinician sees) are only meaningful if the probability itself is trustworthy, not just the argmax decision.
- **How it works:** temperature scaling learns one scalar `T` per model, applied by converting a probability back to a logit, dividing by `T`, and re-applying the sigmoid. `T > 1` softens (de-confidences) predictions that were overconfident; `T < 1` sharpens underconfident ones. `T` is found by minimizing negative log-likelihood on the *validation* split (never test) via bounded scalar optimization. **ECE** is the metric used to check whether it worked: bin predictions by confidence (10 bins), and for each bin compare the average predicted confidence to the actual fraction correct in that bin — ECE is the sample-weighted average of that gap. A well-calibrated model has ECE near 0.

26.9 Population Stability Index (PSI)

- **What it is:** a single number quantifying how much a distribution has shifted between two samples — originally from credit-risk modeling, now widely used for ML model-drift monitoring.
- **Why used here:** a classifier's input distribution can shift after deployment (different users, different phrasing trends, different platforms) in ways that silently degrade its real-world accuracy long before anyone notices — PSI is a cheap, no-ground-truth-needed early-warning signal for exactly that.

- **How it works:** bucket both the "baseline" and "current" probability distributions into the same 5 bins over `[0, 1]`, compute each bin's percentage share of its distribution, then sum `(current% − baseline%) × ln(current% / baseline%)` across bins. This is mathematically a variant of the KL-divergence family — it's large when the two distributions disagree a lot about where their probability mass sits, and near 0 when they're similar. Thresholds used in this codebase: `<0.10` stable, `0.10–0.20` moderate, `≥0.20` critical (see Section 13.4 for what "baseline" and "current" mean concretely in this app).

26.10 Bootstrap Confidence Intervals

- **What it is:** a way to estimate how uncertain a statistic (like "accuracy on this cohort") is, without needing a closed-form formula for its uncertainty — by resampling the data itself.
- **Why used here:** the fairness audit's cohorts are small (32 examples), and a point estimate like "87.5% recall" says nothing about how much that number would wobble on a different sample of 32 similar posts. Reporting a bare percentage without a confidence interval would overstate how precisely known that number actually is.
- **How it works:** from *n* real examples, draw a new sample of size *n* **with replacement** (so some examples appear multiple times, others not at all), compute the metric on that resample, and repeat 1,000 times (`FAIRNESS_BOOTSTRAP_ITERATIONS`). The 2.5th and 97.5th percentiles of those 1,000 resampled metric values form the 95% confidence interval. Two cohorts are flagged as having a statistically detectable gap only when their intervals **don't overlap** — a standard (if conservative) heuristic for "these two numbers are probably genuinely different, not just noise."

26.11 Construct Validity / Confound Residualization

- **What it is:** a technique for checking whether a model's apparent skill at predicting *X* is secretly explained by something else — a *confound* — that happens to correlate with *X* in this particular dataset.
- **Why used here:** it would be easy for a suicide-risk classifier to actually be "just" a sadness/negativity detector, since sad and suicidal text overlap heavily in vocabulary. If that were true, the model's accuracy would look great on this dataset but might fail badly on, say, angry or numb-sounding crisis text that doesn't read as generically "sad."
- **How it works (Dehghan & Ashrafi, 2026's residualization protocol, [Section 3](#3-literature-review--theoretical-grounding)):** score every text on a simple, crisis-language-free negativity lexicon; regress the model's predicted probability on that single negativity score; subtract the regression's prediction from the model's actual prediction to get a **residual** — the part of the model's behavior negativity *can't* explain. If that residual still correlates with the true label, the model has real signal beyond the confound. If it doesn't, the model's accuracy was mostly the confound in disguise.

26.12 Energy-Based Out-of-Distribution (OOD) Detection

- **What it is:** a way to flag inputs that don't resemble anything the model was trained on — e.g. a recipe, a joke, or gibberish — so the system doesn't confidently score something the model was never equipped to judge.

- **Why used here:** a classifier will always output *some* probability, even for input completely outside its training distribution, and it has no built-in way to say "I don't actually know." OOD detection adds that missing signal on top.
- **How it works (Liu et al., 2020):** rather than using the softmax probability itself (which tends to stay overconfident even on nonsense inputs — a known weakness of softmax), this method computes an **energy score** directly from the raw pre-softmax logits: `E(x) = −T · logsumexp(logits / T)`. Lower energy = the model "recognizes" this kind of input; higher = it doesn't. The threshold isn't arbitrary — it's calibrated once, as the 95th percentile of energies the model produced on its own in-distribution validation set, so "anomalous" is defined relative to what this specific model has actually seen.

26.13 MC-Dropout Predictive Uncertainty

- **What it is:** a cheap way to get an *uncertainty estimate* out of a standard neural network without training a second, specialized model for it.
- **Why used here:** calibrated confidence (26.8) reflects one deterministic pass through the network. It doesn't capture a different kind of doubt: "if I ran this through slightly different internal random states, would I get a different answer?" High disagreement across those runs is a second, complementary uncertainty signal.
- **How it works:** dropout (randomly zeroing some neurons) is normally only active *during training*, to prevent overfitting. MC-Dropout keeps it active at inference time too, runs the same input through the network 20 times (`MC_DROPOUT_PASSES`), and looks at how much the 20 outputs disagree. That disagreement is summarized as **predictive entropy** of the averaged probability distribution — entropy is a standard information-theory measure of "spread"/uncertainty in a distribution, maximal when the distribution is a coin-flip (50/50) and zero when it's certain (100/0).

26.14 Change-Point Detection (Binary Segmentation)

- **What it is:** an algorithm for finding the point in a time-ordered sequence of numbers where its statistical behavior abruptly shifts, as opposed to describing the whole sequence with one overall trend.
- **Why used here:** a linear trend line (26.15 below — `compute_trend`) can completely miss a sudden crisis: a person who was stable for five posts and then had one sharp deterioration will show only a *mild* overall upward slope, even though "something changed sharply after post 5" is the far more clinically urgent fact.
- **How it works:** `ruptures`' Binary Segmentation algorithm searches for the single split point that, if you computed the mean probability before and after that point, best explains the sequence (minimizes the total squared error of that two-segment model versus the real data). This project deliberately sets `jump=1, min_size=1` (rather than the library's default `jump=5`) because the default silently has no valid candidate split points on the very short histories (as few as 4–6 posts) this app deals with. To avoid crying wolf on small-sample noise, a candidate change point is only reported if there's a minimum amount of history to begin with (≥4 posts) **and** the magnitude of the before/after mean shift clears a minimum threshold (0.15).

26.15 Linear Trend Detection

- **What it is:** the simplest possible way to summarize "is this going up, down, or flat" — fit a straight line through the data by ordinary least squares and look at its slope.
- **Why used here:** it's the complement to change-point detection (26.14) — a gradual, sustained escalation across many posts (rather than one sharp jump) is exactly what a slope captures and a change-point search might not flag as a single "point."
- **How it works:** `numpy.polyfit(post_index, probability, 1)` fits the best-fit line; its slope, in probability-change-per-post, is compared against fixed thresholds (`±0.05`) to label the trend Escalating / Stable / De-escalating.

26.16 Retrieval-Augmented Generation (RAG)

This one is mostly not an LLM — see the "how it works" bullet below.

- **What RAG normally means:** retrieve relevant documents from a knowledge base, then have a language model generate an answer *grounded in* those retrieved documents, rather than generating purely from its own training memory (which risks hallucination).
- **What this project actually does:** the "retrieval" half is real and fully deterministic — `query_rag_knowledge()` keyword-matches the input against a small hardcoded DSM-5/C-SSRS knowledge base in plain Python, with no learned model involved at all. Historically (before this session) the "generation" half was also just a deterministic string template. **What changed this session** is that the generation half can now optionally be handed to Gemini — but only to *phrase* the already-retrieved, already-decided result, never to *choose* it. See Section 15 for the full design.
- **Why designed this way:** the literature review (Section 3) turned up repeated evidence that LLMs are unreliable when trusted to reason end-to-end about clinical risk. Keeping retrieval and decision-making 100% deterministic, and scoping the LLM to narration only, sidesteps that failure mode entirely rather than trying to mitigate it after the fact.

26.17 WebSockets (Live Monitor)

- **What it is:** a persistent, two-way network connection between browser and server (unlike a normal HTTP request, which opens, gets one response, and closes).
- **Why used here:** the Live Monitor needs to *push* new events to the browser every 6 seconds without the browser having to repeatedly ask "anything new?" (polling) — a WebSocket lets the server send events the instant they happen.
- **How it works:** the frontend opens one `WebSocket` connection to `/api/ws/monitor`; on connect, the server immediately sends the 10 most recent stored events as a `history` message so a freshly-opened tab isn't blank, then streams a new `event` message every time the background feed loop produces one. `MonitorContext.jsx` only opens the socket while the feed is actually running, closing it otherwise, to avoid an idle client endlessly trying to reconnect to a monitor that isn't broadcasting.

26.18 React Context API (Frontend State Management)

- **What it is:** React's built-in mechanism for sharing state across many components without manually passing it down through every intermediate layer ("prop drilling").
- **Why used here:** the app has five clearly-separable pools of shared state (theme, notifications, active tab/model, all analysis data, monitor/live-feed state) that many unrelated components across different tabs all need to read or update — Context avoids threading five sets of props through the entire component tree, without pulling in an external state-management library (Redux, Zustand, etc.) that this project doesn't otherwise need.
- **How it works:** each `XContext.jsx` file creates a `Context` object and a `Provider` component that holds the actual `useState`/`useEffect` logic and exposes it as a single value object; `App.jsx` nests all five providers around the dashboard; any component anywhere inside can call the matching `useX()` hook to read/update that state directly, without it being passed through props at all.

26.19 `asyncio.to_thread` (Backend Concurrency)

- **What it is:** a way to run a normal, blocking (synchronous) Python function inside an `async` FastAPI application without freezing the whole server while it runs.
- **Why used here:** FastAPI's async event loop can only serve other requests *while nothing is blocking it*. Model inference, SHAP computation, and SQLite writes are all synchronous, CPU-bound (or disk-bound) operations that would otherwise stall every other in-flight request on the server for their entire duration.
- **How it works:** `asyncio.to_thread(fn, *args)` hands `fn` off to a background thread pool and immediately frees the event loop to keep handling other requests, resuming the route handler only once that thread finishes. The one documented exception in this codebase is starting the Live Monitor's background task, which must stay on the *main* event loop because it calls `asyncio.create_task()` — a call that requires an already-running event loop, which a background thread doesn't have (this was an actual bug encountered and fixed during development, per the code comment in `monitor.py`).

---

27. Anticipated Q&A

Q: Why train four models instead of just the best one? To make the interpretability/accuracy trade-off an evidence-based comparison rather than an assertion — the linear models give exact, cheap, closed-form explanations; the transformers give ~6 points higher F1 by capturing context bag-of-words can't, at the cost of needing post-hoc explanation methods. The Multi-XAI Studio and Analytics tab let you show both sides on the same data.

Q: How does the fairness audit work, exactly? It does *not* use demographic attributes — there are none in this dataset. It audits three linguistic registers (youth slang, formal, literal/direct) using 16 risk + 16 non-risk scenarios each expressed in all three registers, with bootstrapped 95% confidence intervals per cohort and a minimum-subgroup-size gate (30) before any gap is called statistically significant.

Q: How do you know the models aren't just detecting sadness/negativity in general? The construct-validity auditor, based on Dehghan & Ashrafi (2026)'s residualization protocol, regresses each model's prediction on a generic-negativity lexicon score and checks whether the residual still correlates with the true label. It does — negativity explains under 2.3% of variance in any model's output, while residual–label correlation stays 0.87–0.98.

Q: What stops the Gemini integration from hallucinating clinical advice? It never makes a decision — the DSM-5 keyword retrieval and C-SSRS tier selection are deterministic Python, computed before the LLM is ever called. Gemini is only asked to write a short narrative *explaining* that already-fixed result, under a system instruction that explicitly forbids inventing facts or contradicting the given tier, with PII masked before the request leaves the process, and a template fallback if the call fails for any reason.

Q: Is the "live monitor" real data? It's a simulated feed (5 synthetic users with scripted 6-post storylines, one new post every 6 seconds), but the trend/change-point detection running on top of it is genuinely computed from that logged history in real time — that part of the pipeline is exactly what would run against a real feed.

Q: What would you improve with more time? In priority order: (1) wire up the three orphaned trust/explainability frontend components, (2) replace TF-IDF case search with real sentence embeddings, (3) grow the fairness cohort scenario count past the 30-example significance threshold, (4) add authentication and tighten CORS before any real deployment, (5) replace the multilingual heuristic language-detector/translator with a real translation model.

---

28. Suggested Live Demo Script

60. **Landing page** — headline numbers, dataset, evaluation table, fairness/construct-validity summary, authors, scope. Fastest way to put verified numbers on screen.
61. **Enter dashboard → Diagnostic Assessment** — load a high-risk preset, run analysis, point out the tier badge, calibrated probability, and the word-attribution heatmap.
62. **Multi-XAI Studio** — run the 4-explainer comparison on the same text, show the Pearson convergence matrix.
63. **Clinical Analytics / Model Drift & PSI** — model comparison table, robustness table, PSI dashboard.
64. **Demographic Audit** — walk through the linguistic-register cohorts, the visible-but-uncertified recall gap, and the significance gating logic (this is the single best moment to demonstrate methodological maturity).
65. **Clinical Safety Copilot** — open it from the header, show the DSM-5 RAG Grounding tab, point out the "AI-generated" vs. "Template" badge on the narrative, and explicitly note the Action Dispatcher is simulated.
66. **Live Monitor** — start the feed, let two or three events stream in, show a user's trend sparkline and, if one has occurred, a flagged change point.

End of playbook — X-MHRDS: research prototype for explainable, audited mental-health risk detection. Not a clinical instrument.